CampusVote — Verifiable Student Election System

Supervised, in-person, end-to-end verifiable voting for university and student elections.
NFC student cards for check-in, Flutter booth app, homomorphic encryption for the tally,
and a Solana anchor for a public, tamper-proof audit trail.

> Scope: campus elections (student council, clubs, societies). Not a replacement for
> national or state elections, which only the Election Commission of India can run.

---

1. Goals

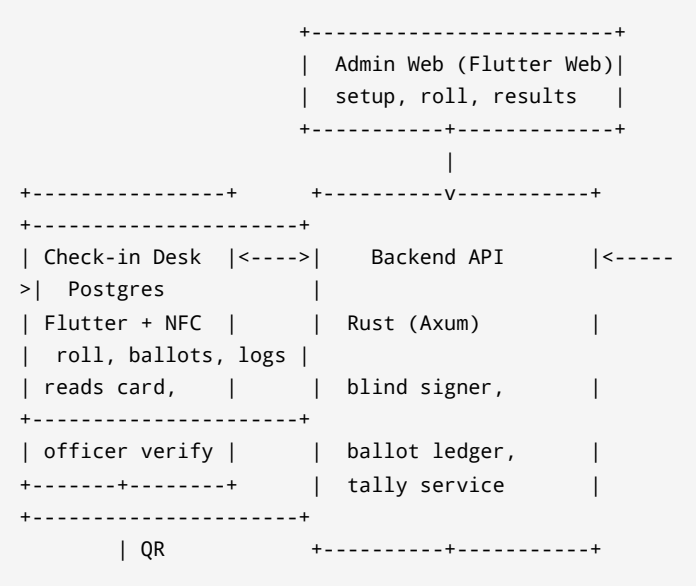
Goal	How it is met
One person, one vote	NFC card auth + roll "voted" flag + on-chain nullifier
Secret ballot	Blind-signed anonymous token, encrypted ballots
Nobody sees partial results	Threshold key: 3 of 5 trustees must cooperate to decrypt
Anyone can verify the result	Public ballot list, zero-knowledge proofs, published tally

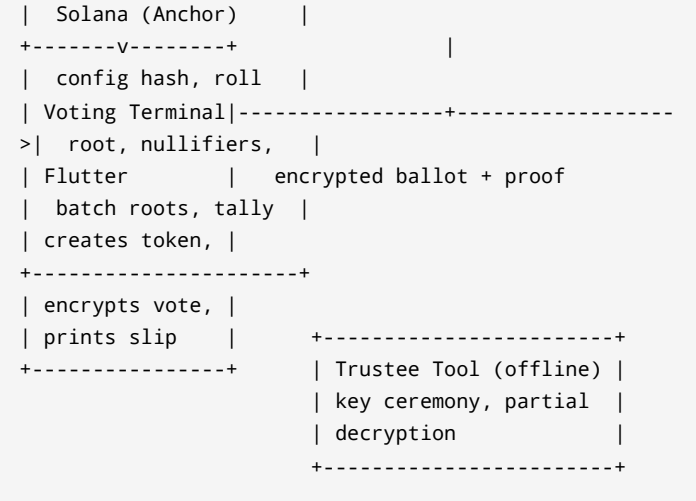
Goal	How it is met
Voter can check their vote was counted	Receipt code (not provable to a third party, so no vote-buying)
Rules cannot change mid-election	Config hash locked on Solana before polling opens
Works if Wi-Fi drops	Offline-first booth app, signed local log, per-booth roll partition
Paper fallback	Optional printed slip kept in a box for audits

Non-goals (v1): remote/phone voting, ranked-choice, anonymous write-ins.

---

2. Architecture





Two devices per booth on purpose:

- Check-in desk knows *who* you are.
- Voting terminal must never know who you are.
- A blind-signed token is the only bridge between them.

---

3. Tech Stack

Client (Flutter)

Need	Choice
Booth + desk apps	Flutter on Android tablets (kiosk / lock-task mode)
Admin dashboard	Flutter Web
NFC	`flutter_nfc_kit` (raw APDU transceive for NTAG 424 DNA)

Need	Choice
State management	`riverpod`
Offline storage	`drift` (SQLite), encrypted with SQLCipher
QR between desk and terminal	`mobile_scanner` + `qr_flutter`
Receipt printing	ESC/POS thermal printer over Bluetooth (`print_bluetooth_thermal`)
Crypto core	Rust library bound with `flutter_rust_bridge` (do **not** write crypto in Dart)
Device integrity	Android Play Integrity API

iOS cannot do arbitrary NFC APDU commands. Use **Android** for booth devices.

Backend

Need	Choice
API	Rust + Axum (or Node + NestJS if you prefer speed of development)
DB	PostgreSQL
Blind signatures	RSA blind signatures (RFC 9474)
Ballot encryption	Elliptic-curve ElGamal on ristretto255 (`curve25519-dalek`)

Need	Choice
Proofs	Chaum-Pedersen / disjunctive proofs (0-or-1 per slot)
Threshold keys	Pedersen DKG + Shamir shares (3-of-5)
Reference implementation	Microsoft **ElectionGuard** (study it, reuse ideas or the SDK)
Audit log	Hash-chained append-only table

Chain

Need	Choice
Program	Anchor (Rust) on Solana
Role	Public timestamp and uniqueness, **not** the security core

Hardware

- Student cards or stickers: **NTAG 424 DNA** (AES, secure dynamic messaging, anti-clone).
  Most campus ID cards are MIFARE Classic or plain UIDs, which are cloneable. Do not trust a bare UID.
- Android tablets x 2 per booth, thermal printer x 1, locked ballot box (paper).
- Trustees: hardware keys (YubiKey or similar) to protect key shares.

---

4. Election Setup

The setup is one JSON file, hashed with SHA-256, and the hash is written to Solana
before polling opens. Changing one character later produces a completely different hash.

```
{
  "election_id": "euc-council-2026",
  "name": "Example University Student Council 2026",
  "positions": [
    {
      "id": "PRES",
      "title": "President",
      "seats": 1,
      "eligible_voters": "all",
      "method": "plurality",
      "candidates": [
        { "code": "PRES-01", "name": "Aarav Sharma" },
        { "code": "PRES-02", "name": "Meera Iyer" },
        { "code": "PRES-03", "name": "Rohan Deshmukh"
},
        { "code": "PRES-04", "name": "Isha Kulkarni"
},
        { "code": "PRES-05", "name": "Kabir Singh" }
      ]
    },
    {
      "id": "TRES",
      "title": "Treasurer",
      "seats": 1,
      "eligible_voters": "all",
      "method": "plurality",
      "candidates": [
        { "code": "TRES-01", "name": "Neha Patil" },
        { "code": "TRES-02", "name": "Vikram Joshi" },
        { "code": "TRES-03", "name": "Sana Khan" }
      ]
    }
  ],
  "rules": {
```

```
    "votes_per_position": 1,
    "abstain_allowed": true,
    "tie_break": "re-vote between tied candidates",
    "revote_allowed": false
  },
  "voting_window": {
    "opens": "2026-11-05T09:00:00+05:30",
    "closes": "2026-11-05T17:00:00+05:30"
  },
  "trustees": { "total": 5, "threshold": 3 },
  "voter_roll_root": "<Merkle root of the roll>"
}
```

Config hash (illustrative): `9f2c71…a41b` → stored in the `ElectionConfig` account on Solana.

---

5. Roles and Key Ceremony

Role	Can do	Cannot do
Registrar	Upload voter roll	See votes
Election officer	Create setup, run check-in desk	Decrypt, edit config after lock
Booth poll officer	Verify a student's face and card	See who voted for whom
Trustee (x5)	Hold one key share	Decrypt alone
Observer / candidate agent	Watch booths, read public ledger	Change anything

Key ceremony (before polling):

1. 5 trustees run DKG on their own devices. The election public key is produced.
2. Nobody ever holds the full private key. Each holds one share.
3. The public key is published and included in the config.
4. After polls close, any 3 trustees combine their partial decryptions of the **aggregate only**.

---

6. Voting Flow (per voter)

1. **Check-in.** The student taps their NFC card on the desk tablet.
   - The app verifies the card's cryptographic signature (NTAG 424 DNA SDM), not just the UID.
   - The roll lookup shows name, photo, constituency and "not yet voted".
   - The officer visually verifies the person.
2. **Token request.** The voting terminal creates a random token `t`, blinds it, and shows the blinded value as a QR.
3. **Signing.** The desk scans the QR. The backend signs the blinded value and, in the **same database transaction**, marks the voter as voted. The desk shows the signature as a QR.
4. **Unblinding.** The terminal scans it, removes the blinding, and holds `t` plus a valid signature. The signer never saw `t`, so the token cannot be linked to the student.
5. **Voting.** The student picks one candidate per position (or abstains). The terminal encrypts each slot and attaches proofs.
6. **Cast or spoil.** The terminal shows a receipt code. The student chooses:

- *Cast:* the ballot is submitted.
- *Spoil / challenge:* the terminal reveals what it encrypted, the ballot is discarded and the student re-votes. This catches a cheating terminal (Benaloh challenge).

7. **Submission.** The ballot goes to the backend with `t` and its signature. The backend verifies, records `hash(t)` as a nullifier, stores the ballot, and sends the nullifier to Solana.
8. **Cleanup.** The terminal erases `t` and resets. An optional paper slip prints and drops into the box.

---

7. Worked Example With Real Numbers

> **Toy parameters.** The numbers are tiny so you can verify them on a calculator.
> Production uses 256-bit elliptic curves. The math is identical.

7.1 Setup

- Public numbers: modulus **p = 23**, generator **g = 5**
- Trustees' secret key: **x = 6**
- Public key: **h = 5^6 mod 23 = 8**

Table of g^k mod 23 (used later to turn a result back into a count):

count k	0	1	2	3	4	5	6	7	8	9	10
g^k mod 23	1	5	2	10	4	20	8	17	16	11	9

7.2 Candidates and codes

President

Code	Candidate
PRES-01	Aarav Sharma
PRES-02	Meera Iyer
PRES-03	Rohan Deshmukh
PRES-04	Isha Kulkarni
PRES-05	Kabir Singh

Treasurer

Code	Candidate
TRES-01	Neha Patil
TRES-02	Vikram Joshi
TRES-03	Sana Khan

7.3 Voters (the roll) and what each chose

Roll codes are used only at check-in. After the token step, ballots are not linked to them.

Roll code	Voter	President	Treasurer
EUC-1001	Ananya Rao	PRES-02 Meera	TRES-01 Neha

Roll code	Voter	President	Treasurer
EUC-1002	Dev Malhotra	PRES-02 Meera	TRES-01 Neha
EUC-1003	Priya Nair	PRES-01 Aarav	TRES-02 Vikram
EUC-1004	Sahil Gupta	PRES-04 Isha	TRES-01 Neha
EUC-1005	Tanvi Bhosale	PRES-02 Meera	TRES-03 Sana
EUC-1006	Arjun Reddy	PRES-03 Rohan	TRES-02 Vikram
EUC-1007	Zoya Ansari	PRES-05 Kabir	TRES-01 Neha
EUC-1008	Nikhil Pawar	PRES-02 Meera	TRES-02 Vikram
EUC-1009	Riya Mehta	PRES-01 Aarav	TRES-01 Neha
EUC-1010	Omkar Jadhav	PRES-04 Isha	TRES-03 Sana

7.4 How one ballot is encrypted

Each position is **one-hot encoded**: one slot per candidate, `1` for the chosen one, `0` for the rest.
Each slot is encrypted separately with its own random number `r`:

```
ciphertext = ( c1, c2 ) = ( g^r mod p ,  g^m * h^r mod p )      m = 0 or 1
```

Ananya (EUC-1001), President ballot: she votes Meera.

Slot	m	r	c1 = 5^r mod 23	c2 = 5^m * 8^r mod 23	Ciphertext
PRES-01 Aarav	0	3	10	1 * 6 = 6	(10, 6)
PRES-02 Meera	1	4	4	5 * 2 = 10	(4, 10)
PRES-03 Rohan	0	2	2	1 * 18 = 18	(2, 18)
PRES-04 Isha	0	5	20	1 * 16 = 16	(20, 16)
PRES-05 Kabir	0	1	5	1 * 8 = 8	(5, 8)

Ananya, Treasurer ballot: she votes Neha.

Slot	m	r	c1	c2	Ciphertext
TRES-01 Neha	1	2	2	5 * 18 = 90 mod 23 = 21	(2, 21)
TRES-02 Vikram	0	3	10	6	(10, 6)
TRES-03 Sana	0	1	5	8	(5, 8)

Note that (10, 6) appears in both ballots for a "0" slot but with different meaning in a real run;
real `r` values are huge and random, so identical votes never look identical.

What gets stored: only the ciphertext pairs plus proofs. The ledger cannot tell who Ananya voted for.

Decrypting one slot (only to show it works, never done in real life):
Meera slot (4, 10): `4^6 mod 23 = 2`. Then `10 / 2 = 5 = 5^1`, so the vote is `1`. ✔
Aarav slot (10, 6): `10^6 mod 23 = 6`. Then `6 / 6 = 1 = 5^0`, so the vote is `0`. ✔

7.5 Counting without opening any ballot

Multiply the ciphertexts of the same candidate across all voters, component by component.
Here is **Sana Khan (TRES-03)** across all 10 voters. Sana got a "1" from Tanvi (EUC-1005) and Omkar (EUC-1010).

Voter	m (vote for Sana)	r	Ciphertext (c1, c2)
EUC-1001	0	1	(5, 8)
EUC-1002	0	2	(2, 18)
EUC-1003	0	3	(10, 6)
EUC-1004	0	4	(4, 2)
EUC-1005	**1**	5	(20, 11)
EUC-1006	0	6	(8, 13)
EUC-1007	0	7	(17, 12)

Voter	m (vote for Sana)	r	Ciphertext (c1, c2)
EUC-1008	0	8	(16, 4)
EUC-1009	0	9	(11, 9)
EUC-1010	1	11	(22, 5)

Multiply all `c1` values mod 23: **18**
Multiply all `c2` values mod 23: **16**
Combined ciphertext: **(18, 16)**

Trustees decrypt only this combined pair:

1. `18^6 mod 23 = 8` (using the secret key 6)
2. `16 / 8 mod 23 = 2`
3. Look up in the table: `5^2 mod 23 = 2`, so the total is **2 votes for Sana.** ✔

The same process runs for every candidate. In production the trustees each compute a partial
decryption with their share, and any 3 combine them. Nobody ever holds `x`.

7.6 Final results

Code	Candidate	Combined decrypts to	Votes
PRES-01	Aarav Sharma	2	**2**
PRES-02	Meera Iyer	4	**4**
PRES-03	Rohan Deshmukh	5	**1**

Code	Candidate	Combined decrypts to	Votes
PRES-04	Isha Kulkarni	2	**2**
PRES-05	Kabir Singh	5	**1**
		Total President votes	**10**

President: Meera Iyer (PRES-02), 4 of 10 votes.

Code	Candidate	Combined decrypts to	Votes
TRES-01	Neha Patil	20	**5**
TRES-02	Vikram Joshi	10	**3**
TRES-03	Sana Khan	2	**2**
		Total Treasurer votes	**10**

Treasurer: Neha Patil (TRES-01), 5 of 10 votes.

Sanity checks the system runs automatically:

- Votes per position = number of ballots cast = 10 (plus abstentions, if any).
- Total check-ins (10) = total nullifiers (10) = total ballots (10).
- Every ballot's proof confirms each slot is 0 or 1 and exactly one slot per position is 1.

7.7 Receipts

After casting, the student gets a code derived from the hash of their ballot ciphertexts,
for example `A91F-3C07`. Anyone can look the code up on the public ballot list and confirm the
ballot was recorded unchanged. The receipt does **not** reveal the vote, so it cannot be used to
prove how you voted to a vote-buyer. (Codes here are illustrative, not real hashes.)

---

8. Double-Voting and Fraud Defenses

Attack	Defense
Cloned card	NTAG 424 DNA rolling CMAC, server verifies each tap. Plain UID is rejected
Same student votes twice at one booth	Roll flag set in the same DB transaction as signing (`SELECT ... FOR UPDATE`)
Same student votes at two booths	Central roll flag online; per-booth roll partition offline
Reusing a token	`hash(t)` nullifier is unique in the DB **and** as a Solana PDA (second creation fails)
Forged token	RSA blind signature verified by the backend
Ballot stuffing by an admin	Total check-ins = nullifiers = ballots, all publicly auditable
Ballot box tampering after the fact	Batch Merkle roots anchored on Solana every N minutes

Attack	Defense
Rules changed mid-election	Config hash on-chain
Early peeking at results	Threshold key, 3 of 5 trustees
Cheating voting terminal	Benaloh cast-or-spoil challenge + paper slip audit
Stolen booth tablet	Kiosk lock, Play Integrity attestation, per-device keys revocable on-chain
Coercion / vote buying	Supervised booths, receipts that prove nothing

---

9. Data Model (PostgreSQL)

```
elections(id, name, config_json, config_hash, status,
opens_at, closes_at)
positions(id, election_id, code, title, seats, method)
candidates(id, position_id, code, name)
voters(id, election_id, roll_code, name, constituency,
card_key_ref,
       voted boolean default false, booth_id)
cards(id, voter_id, chip_uid_hash, status)
-- active / revoked / replaced
checkins(id, voter_id, booth_id, officer_id,
ts_minute)  -- coarse timestamp on purpose
ballots(id, election_id, position_id, ciphertexts
jsonb, proofs jsonb,
        nullifier unique, receipt_hash, booth_id,
cast_at_minute)
trustees(id, election_id, name, pubkey,
share_commitment)
```

```
tally_results(id, position_id, candidate_id,
combined_ct jsonb, partials jsonb, count)
audit_log(id, prev_hash, entry jsonb, entry_hash)
-- hash chain
booth_devices(id, role, device_pubkey, status)
```

Do **not** store a link between `voters.id` and `ballots.id`. Round timestamps to the minute and publish ballots in shuffled batches to weaken timing correlation.

Solana accounts (Anchor)

Account	Seeds	Stores
`ElectionConfig`	`["election", election_id]`	config_hash, roll_root, window, status, tally_hash
`Nullifier`	`["null", election_id, nullifier]`	existence only; creating it twice fails
`BatchRoot`	`["batch", election_id, index]`	Merkle root of ballot hashes in that batch
`BoothAuthority`	`["booth", device_pubkey]`	active flag

Blind-signature verification happens in the backend (RSA-2048 verification is too heavy for on-chain). The chain enforces **uniqueness, timing, and immutability**, not signature checking.

---

10. API (v1)

Method	Path	Who	Purpose
POST	`/elections`	officer	create draft setup
POST	`/elections/:id/lock`	officer	hash config, write to chain, freeze
POST	`/elections/:id/roll`	registrar	upload roll, compute Merkle root
POST	`/checkin/verify-card`	desk	verify NFC SDM payload, return voter info
POST	`/checkin/sign-token`	desk	blind-sign + mark voted (one transaction)
POST	`/ballots`	terminal	submit encrypted ballot, token, signature, proofs
GET	`/ballots?batch=n`	public	list ballots and receipt

Method	Path	Who	Purpose
			hashes
GET	`/receipts/:code`	public	confirm a ballot was recorded
POST	`/tally/partial`	trustee	submit partial decryption of aggregates
GET	`/results/:id`	public	final tally + proofs

---

11. Flutter Project Layout

```
campusvote/
├─ apps/
│  ├─ booth_desk/        # NFC check-in, QR sign flow
│  ├─ booth_terminal/    # ballot UI, encrypt, print
slip
│  ├─ admin_web/         # setup, roll, results,
observer view
│  └─ trustee_tool/      # offline key ceremony +
partial decrypt
├─ packages/
│  ├─ crypto_core/       # Rust: ElGamal, proofs,
blind sig, Merkle (via flutter_rust_bridge)
│  ├─ nfc_ntag424/       # APDU + SDM verification
│  └─ ui_kit/
├─ backend/              # Axum API, tally service
├─ programs/campusvote/  # Anchor program
└─ docs/
```

NTAG 424 DNA tap flow:

1. Card is personalised once by the registrar (per-card AES key derived from a master key + chip UID).
2. On each tap the chip emits a URL-like payload: `uid + counter + CMAC`.
3. Backend recomputes the CMAC and rejects a replayed or stale counter.

---

12. Offline Mode

- Each booth gets a signed **roll slice** (only the voters assigned to that booth or department) before polling.
- The desk can sign tokens locally with a booth-specific blind-signing key, logging every event in an append-only, hash-chained local DB.
- On reconnect the log syncs. Any voter appearing at two booths is flagged immediately.
- Roll slices are **disjoint**, so cross-booth double voting offline is impossible by construction.

---

13. Build Order

1. **Phase 1: Core, no NFC.** Config, roll + Merkle root, crypto core (encrypt, prove, aggregate, threshold decrypt), tally, CLI tests against this README's numbers.
2. **Phase 2: Booth apps.** Desk + terminal Flutter apps, QR token flow, backend ballot ledger, receipts.
3. **Phase 3: NFC.** NTAG 424 DNA personalisation, SDM verification, card revoke / replace.

4. **Phase 4: Chain.** Anchor program, nullifier PDAs, batch roots, config lock.
5. **Phase 5: Hardening.** Offline mode, Benaloh challenge, kiosk lock, paper slips, load test.
6. **Phase 6: Pilot.** A small club election with real observers, then a full department.

14. Testing

- Unit tests: encryption, aggregation and decryption against the numbers above.
- Property tests: random elections, tally must equal plaintext counts.
- Fault injection: cloned cards, replayed tokens, dropped network mid-vote, duplicate submissions.
- Independent verifier: a standalone script that downloads the public data and checks all proofs.
- Dry-run election with volunteers before the pilot.

15. Known Limitations

- Timing correlation between check-in and ballot submission (mitigated, not eliminated).
- A compromised terminal sees votes in plaintext; the cast-or-spoil challenge and paper slips detect it.
- Needs a trusted registrar for an accurate roll and honest card issuance.
- Receipts prevent vote-selling proof but not shoulder-surfing inside a booth; booth layout matters.

- This is not a legal substitute for any election your institution's rules require a specific process for. Get the election committee's written approval first.